Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Кафедра прикладної математики

Звіт

про виконання лабораторної роботи №3
з курсу «Основи інженерії даних»

Виконала:
студентка групи ПМІД-11
Мазур Д. М.

Перевірив:
Алєксєєв В. І.

Львів — 2026

Тема: Побудова відмовостійкого конвеєра даних.

Мета роботи: Розробити та реалізувати конвеєр даних, стійкий до збоїв, який забезпечує цілісність даних шляхом перевірки та надійної обробки помилок. Ознайомитися з ідемпотентністю, обробкою помилок та теорією надійності в практичному контексті.

Постановка задачі: Побудувати конвеєр для отримання та перевірки ненадійних потокових даних датчиків.

Завдання до роботи:

1. Створити генератор даних, який вводить помилки.
2. Створити компоненти конвеєра для отримання, перевірки та обробки помилок.

Хід виконання роботи:

1. **Архітектура конвеєра.** Код розміщено в папці `lab3_db`: `data_generator.py` — ненадійне джерело, `pipeline.py` — валідація, дедуплікація та запис, `process.py` — отримання даних з повторними спробами, `metrics.py` — звіт про роботу. Потік даних показано на рис. 1.

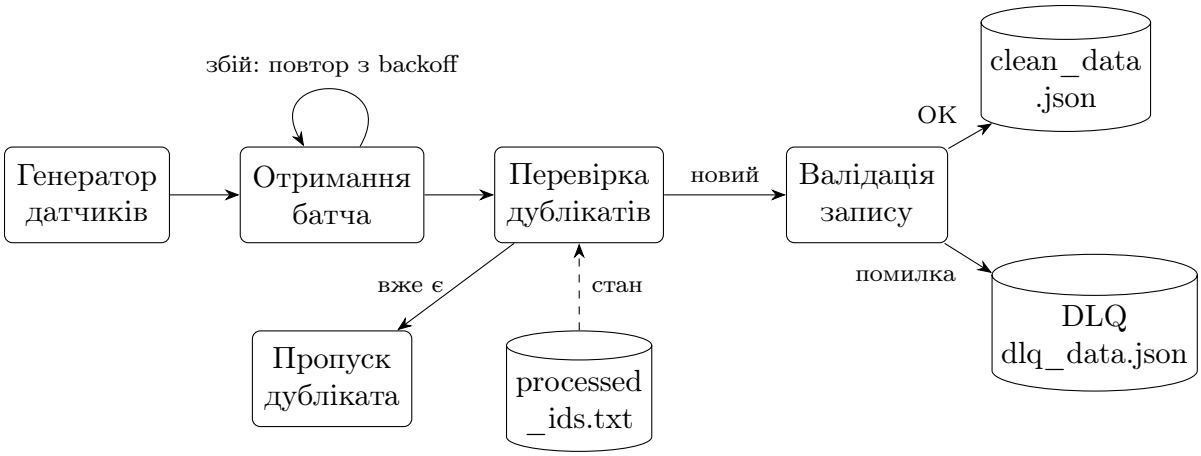


Рис. 1: Потік даних у відмовостійкому конвеєрі

2. **Генератор ненадійних даних.** Клас `UnreliableDataGenerator` створює записи датчиків з полями `record_id` (UUID), `timestamp`, `sensor_id`, `temperature`, `humidity`. Він імітує три класи збоїв реальних потокових систем (табл. 1). Дублікат є точною копією вже надісланого запису, що відповідає семантиці доставки *at-least-once*.

Табл. 1: Типи збоїв, які вводить генератор

Тип збою	Частка	Приклади
Пошкоджений запис	40%	відсутнє поле; `temperature = None`; `"INVALID_TEMP"`; значення поза межами (999.99); некоректний `timestamp`
Дублікат	20%	повторна доставка вже надісланого запису (той самий `record_id` і значення)
Недоступність джерела	30%	виняток `SourceUnavailableError` під час отримання батча

3. **Отримання з повторними спробами.** Тимчасові збої джерела обробляються функцією `fetch_with_retry`. Вона виконує до 4 спроб з експоненційною затримкою 0.2, 0.4 і 0.8 с. Якщо всі спроби невдалі, батч фіксується в метриках як втрачений, і конвеєр продовжує роботу, а не аварійно завершується.

```
for attempt in range(1, MAX_RETRIES + 1):
    try:
        return generator.generate_stream(batch_size=batch_size)
    except SourceUnavailableError as e:
        if attempt == MAX_RETRIES:
            pipeline.record_fetch_failure()
            return None
        time.sleep(BASE_BACKOFF_SEC * 2 ** (attempt - 1))
```

4. **Валідація.** Метод `validate_record` перевіряє, що запис є словником, містить усі обов'язкові поля, має коректні `record_id`, `sensor_id` і ISO-`timestamp`. Температура і вологість мають бути числами (тип `bool` явно виключено) і лежати в допустимих межах: $-50 \dots 100$ °C та $0 \dots 100\%$. Записи, які не пройшли перевірку, не відкидаються, а потрапляють у *Dead Letter Queue* (`dlq_data.json`) разом з причиною відмови та часом. Їх можна проаналізувати й обробити повторно.

5. **Ідемпотентність.** Ідентифікатори всіх оброблених записів, як чистих, так і відправлених у DLQ, зберігаються у файлі стану `processed_ids.txt`. Запис з уже відомим `record_id` пропускається. Тому повторна доставка, зокрема після перезапуску конвеєра, не створює дублікатів у сховищі. Поєднання доставки at-least-once з дедуплікацією дає семантику *effectively-once*.

6. **Надійний запис.** Для стійкості до аварійного завершення застосовано два прийоми:

   - *порядок «дані, потім стан»*: ID фіксуються у стані лише після успішного запису даних. Якщо процес впаде між цими кроками, запис буде оброблено повторно, а не втрачено;
   - *атомарний запис*: JSON спочатку записується в тимчасовий файл, а потім замінює основний через `os.replace`. Збій посеред запису не залишає пошкодженого файлу. Якщо файл усе ж пошкоджено, він не перезаписується мовчки, а зберігається як резервна копія.

7. **Метрики та результати.** Конвеєр веде лічильники в `pipeline_metrics.json`. Скрипт `metrics.py` звіряє їх із фактичним вмістом сховища: перевіряє, що кількість отриманих записів дорівнює сумі чистих, DLQ і дублікатів. Результати запуску з 10 батчів по 10 записів і повторною доставкою 5 записів наведено в табл. 2 та на рис. 2.

Висновки: У ході виконання лабораторної роботи №3 побудовано відмовостійкий конвеєр для ненадійних потокових даних датчиків. Генератор вводить пошкоджені записи, дублікати й тимчасові відмови джерела. Конвеєр опрацював усі ці збої: 105 отриманих записів розподілено на 39 чистих, 32 записи в DLQ і 34 пропущені дублікати. Сума збігається з кількістю отриманих, тобто жоден запис не втрачено. Шість тимчасових відмов джерела було подолано повторними спробами з експоненційною затримкою, без жодного втраченого батча. Ідемпотентність на основі `record_id`, порядок «дані, потім стан» і атомарний запис файлів гарантують, що повторний запуск або збій посеред обробки

Табл. 2: Метрики роботи конвеєра

Метрика	Значення
Отримано записів	105
Чисті записи	39
Записи у DLQ	32
Пропущені дублікати	34
Повторні спроби отримання	6
Втрачені батчі	0
Частка дублікатів	32.4%
Частка валідних (з унікальних)	54.9%

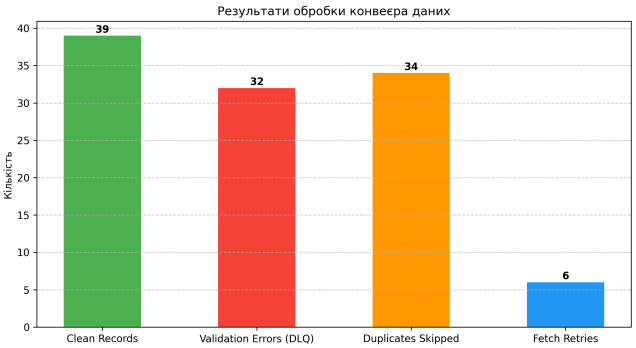


Рис. 2: Розподіл результатів обробки

не призводять ні до дублікатів, ні до втрати чи пошкодження даних. Dead Letter Queue зберігає некоректні записи разом із причиною відмови, що дає змогу аналізувати якість джерела, а не просто відкидати дані.